

Chapter 9

Audio Processing and Recognition

Audio is not a visual modality, but many audio problems become closely related to computer vision once a waveform is converted into a **time-frequency representation**. A spectrogram is a two-dimensional tensor, so convolutional networks and transformers can be adapted to tasks such as sound classification, event detection, and speech recognition.

However, a spectrogram is not an ordinary image. Its horizontal axis represents time, its vertical axis represents frequency, and its values represent magnitude or power. A horizontal translation corresponds to a sound occurring later, while a vertical translation changes its frequency content. These semantics should be considered when choosing augmentations, pooling operations, and model architectures.

This chapter introduces the audio representations needed for deep learning. We start from a digital waveform, construct spectrograms and mel spectrograms, discuss common augmentation techniques, and finish with automatic speech recognition and Connectionist Temporal Classification (CTC).

9.1 From Waveforms to Log-Mel Spectrograms

In practice, most audio deep-learning pipelines begin with a waveform and convert it into a two-dimensional representation:

$$\underbrace{(B, C, L)}_{\text{waveform}} \longrightarrow \underbrace{(B, C, F, T)}_{\text{spectrogram}} \longrightarrow \underbrace{(B, C, R, T)}_{\text{log-mel spectrogram}} .$$

Here B is the batch size, C the number of audio channels, L the number of waveform samples, F the number of frequency bins, T the number of time frames, and R the number of mel bands.

9.1.1 Digital Waveforms

A digital waveform is a sequence of amplitude values recorded at a fixed **sample rate** f_s . For example, one second of mono audio sampled at 16 kHz contains 16k values and has shape (1, 16k). Stereo audio has two channels and would have shape (2, 16,000). A batch would add a leading dimension.

The sample rate also determines the highest frequency that can be represented: half the sample rate, known as the **Nyquist frequency**. Therefore, audio sampled at 16 kHz can represent frequencies up to 8 kHz. A 10 kHz tone cannot be represented correctly at this sample rate and may appear as a false 6 kHz tone. This effect is called **aliasing**. To avoid it, frequencies above the Nyquist limit are filtered out before recording or downsampling.

Example: Understanding a waveform tensor

Suppose a batch contains $B = 32$ mono recordings. We assume that every recording has been cropped or padded to two seconds and uses a sample rate of 16 kHz. The number of samples in each recording is therefore

$$L = 2 \text{ s} \times 16\text{k samples/s} = 32\text{k samples}.$$

The complete batch has shape

$$(B, C, L) = (32, 1, 32\text{k}).$$

This tensor describes amplitude over time; it does not yet separate the frequencies present in the audio.

9.1.2 From Frames to a Spectrogram

The frequency content of audio changes over time. A Fourier transform of the complete waveform tells us which frequencies are present, but not when they occur. The **Short-Time Fourier Transform (STFT)** solves this problem by dividing the waveform into short, usually overlapping **frames** and analysing each frame separately. It uses three main parameters:

- **Window length** W : number of samples in each frame.
- **Hop length** H : distance between the starts of consecutive frames. When $H < W$, the frames overlap.
- N_{FFT} : size of the Fourier transform applied to each frame. It is commonly equal to or greater than W .

For a waveform of L samples without padding, the number of frames is

$$T = \left\lfloor \frac{L - W}{H} \right\rfloor + 1.$$

Each frame is multiplied by a smooth window function, such as a Hann window, to reduce boundary effects. The Fast Fourier Transform (FFT) then converts that frame from amplitude over time into a frequency spectrum. The magnitude or power spectra are stacked as columns to form a **spectrogram**: time is shown horizontally, frequency vertically, and colour represents magnitude or power.

For a real waveform, the number of non-redundant frequency bins is

$$F = \left\lfloor \frac{N_{\text{FFT}}}{2} \right\rfloor + 1.$$

A longer window distinguishes nearby frequencies better, while a shorter window locates brief events more precisely. This is the practical **time-frequency trade-off**. A smaller hop produces more overlapping frames and follows changes more densely, but also increases the number of columns and the computation. Increasing N_{FFT} adds frequency bins; if it is larger than the window, however, the added zero-padding does not create new information.

Example: A common speech configuration

At 16 kHz, a 25 ms window contains $W = 400$ samples and a 10 ms hop contains $H = 160$ samples. Consecutive frames therefore overlap by 240 samples. For one second of audio, $L = 16,000$, so without padding

$$T = \left\lfloor \frac{16\text{k} - 400}{160} \right\rfloor + 1 = 98.$$

Choosing $N_{\text{FFT}} = 400$ produces $F = \lfloor 400/2 \rfloor + 1 = 201$ frequency bins. The spectrogram shape is thus

$$(C, F, T) = (1, 201, 98).$$

Some libraries pad the waveform before framing, so the exact value of T may differ slightly.

9.1.3 Log-Mel Spectrograms

A linear spectrogram allocates equally spaced bins in Hertz. Human hearing is more sensitive to frequency differences at low frequencies, so a **mel filterbank** combines the linear bins into R perceptually spaced bands. A logarithm or decibel conversion then compresses the large range of power values. The result is a **log-mel spectrogram**. Figure 9.1 summarises this conversion.

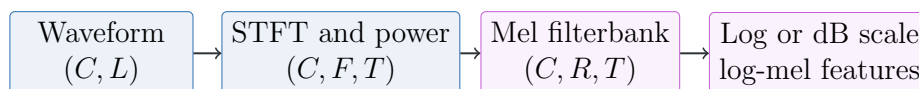


Figure 9.1: Construction of a log-mel spectrogram.

The number of mel bands R controls the vertical resolution of the log-mel spectrogram. A small R produces a compact and smoother representation, but it may merge nearby frequency

patterns. A larger R preserves finer detail at the cost of a larger input and more computation. However, increasing R cannot recover information that is absent from the original spectrogram; it is only useful when there are enough frequency bins F to support those mel bands.

For example, applying $R = 80$ mel filters to the previous spectrogram changes its shape from $(1, 201, 98)$ to $(1, 80, 98)$. The time dimension is preserved. Figure 9.2 compares several values of N_{FFT} and R .

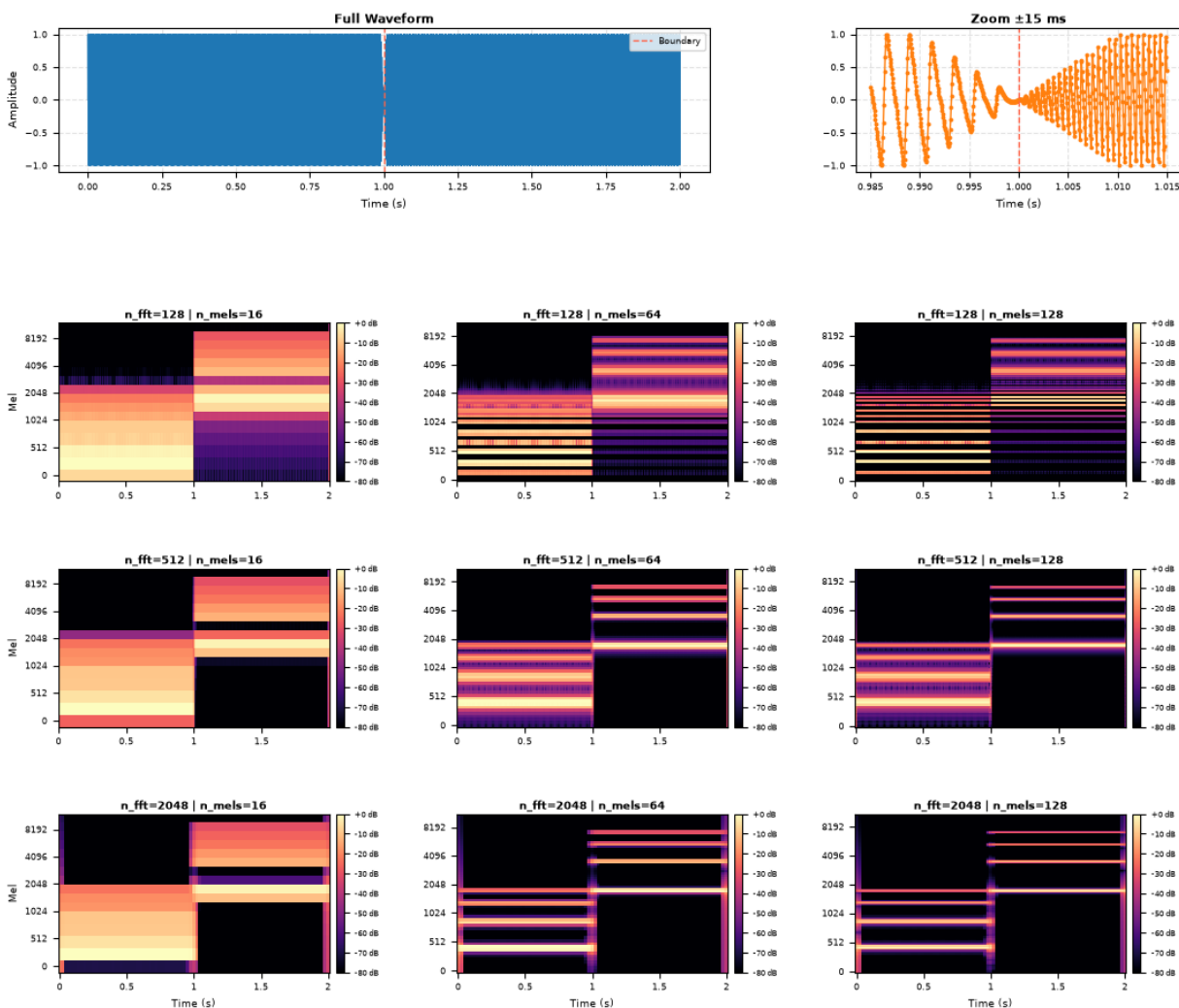


Figure 9.2: A waveform and its log-mel spectrograms for different FFT sizes and numbers of mel bands.

Note

The appropriate sample rate, window, hop, and number of mel bands depend on the task. Speech, music, environmental audio, and very short events may require different values.

9.2 Audio Data Augmentation

Audio augmentation increases training diversity and can improve robustness. An augmentation should preserve the target label. For example, a transformation appropriate for sound classification may be incorrect for pitch estimation or temporally precise event detection.

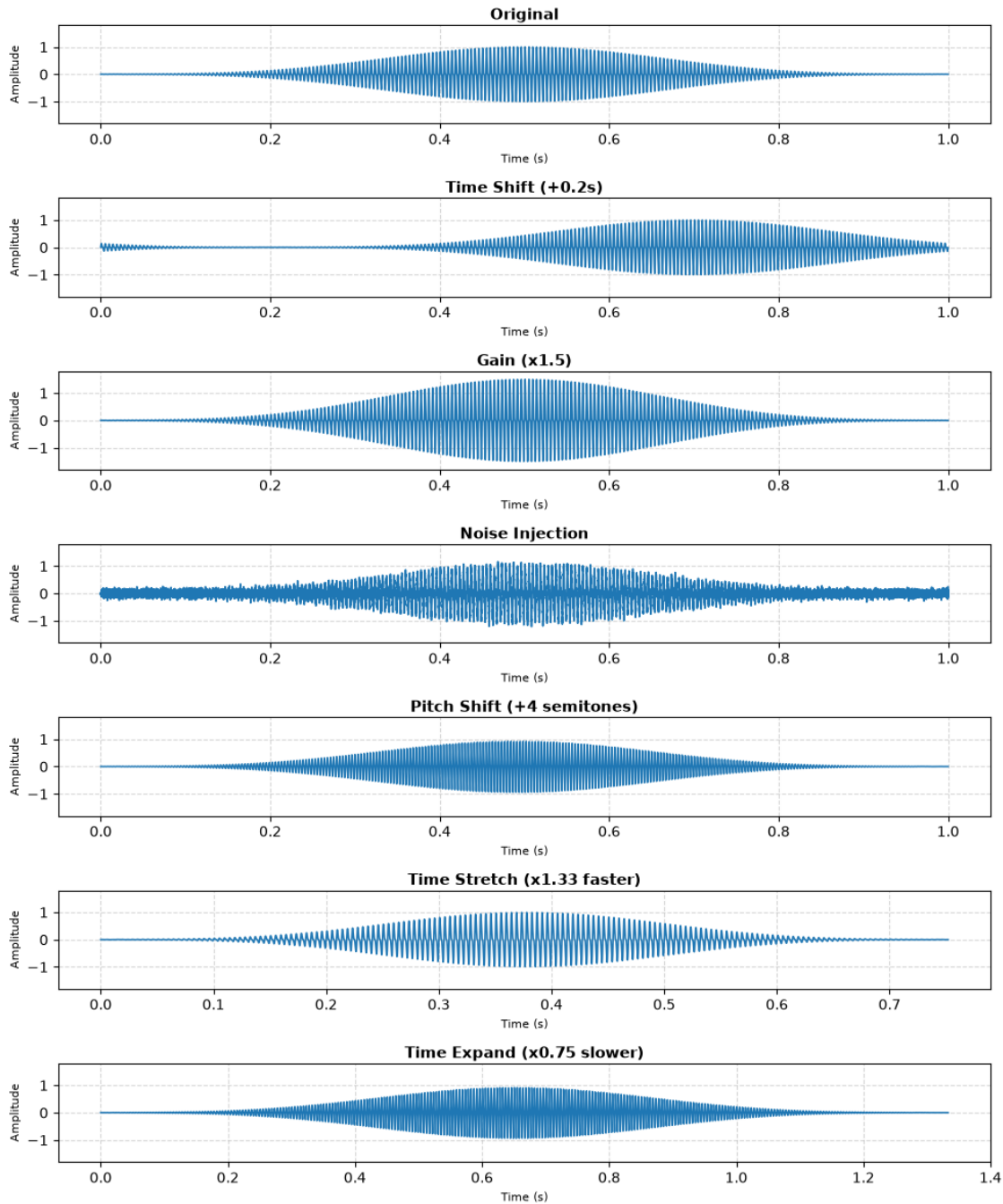


Figure 9.3: Data augmentation techniques for audio.

Figure 9.3 shows common transformations applied before feature extraction:

- **Time shift:** move the waveform forwards or backwards and pad the exposed region. Circular wrapping is easy to implement but introduces an artificial connection between the end and beginning of a recording.
- **Gain:** multiply the waveform by a random factor, taking care to avoid clipping.
- **Noise injection:** add background noise, often at a selected signal-to-noise ratio rather than using an arbitrary fixed amplitude.
- **Pitch shift:** raise or lower pitch without changing duration. It should not be used when pitch itself determines the label.
- **Time stretching:** make the recording faster or slower without changing pitch. It may simulate different speaking rates or musical tempi, but it also changes time annotations.

Example: Choose augmentations according to the task

For clip-level dog-bark classification, a small time shift and moderate background noise normally preserve the label “dog bark”. For sound-event localisation, the same time shift must also move the event timestamp. For pitch estimation, pitch shifting changes the target itself and should not be applied without updating the label.

9.3 Deep Learning with Audio Representations

9.3.1 From Spectrograms to Neural Networks

A spectrogram permits the reuse of many vision architectures:

- A **2D CNN** learns local time-frequency patterns such as harmonics, onsets, and frequency sweeps.
- A **1D CNN** may convolve over the raw waveform or along the time axis of an extracted feature sequence.
- An **RNN** models temporal dependencies after a CNN or feature extractor.
- A **transformer** treats frames or groups of time-frequency bins as tokens and uses attention to model longer-range context.

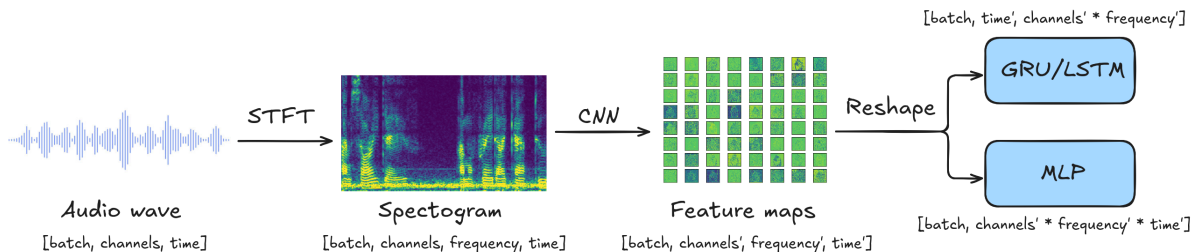


Figure 9.4: Typical deep learning pipeline for audio.

The model input is often (B, C, F, T) . Here C represents the audio channels, F the frequency bins, and T the time frames. Thus, mono audio normally has one spectrogram channel and stereo audio has two. Figure 9.4 shows one possible pipeline.

The CNN converts the input into feature maps of shape (B, C', F', T') . For a single prediction about the complete recording, these features can be pooled or flattened and passed to an MLP. For a sequence task, each time step can instead be represented by a vector of size $C'F'$ and processed by an RNN or a transformer.

Warning: Use the axes according to the task

A spectrogram can be processed as a 2D image, but its axes are not interchangeable. For dog-bark classification, the exact time of the bark may not matter, so pooling over time is useful. For speech recognition, the order and timing of sounds are essential, so the model must preserve a temporal sequence. Similarly, moving a pattern vertically changes its frequencies or pitch; it is not equivalent to moving an object up or down in a photograph.

9.3.2 Common Audio Tasks

Table 9.1 summarises common tasks in audio deep learning.

Task	Typical output	Example
Audio classification	One or several labels per recording	Music genre, environmental sound, or emotion
Sound event detection	Labels and temporal locations	Detecting alarms or machine faults in a long recording
Speaker recognition	Speaker identity or similarity score	Identification and verification
Automatic speech recognition	Sequence of characters, subwords, or words	Transcribing spoken language
Audio generation and enhancement	Waveform or time-frequency representation	Text-to-speech, denoising, or source separation

Table 9.1: Examples of audio learning tasks.

9.4 Automatic Speech Recognition

Automatic Speech Recognition (ASR) receives speech and returns its transcript. For example, its input may be a one-second recording and its output the text “hello”.

A simple ASR pipeline has three parts:

1. Convert the waveform into a sequence of log-mel frames.

2. Use an encoder, such as a CNN followed by an RNN or transformer, to extract contextual features from those frames.
3. Convert the encoder features into text tokens using CTC or an attention-based decoder.

The last step is difficult because the audio frames and text tokens are not already aligned.

9.4.1 The Alignment Problem

Speech and text have different lengths. With a 10 ms hop, one second of audio may contain 100 feature frames, but “hello” contains only five characters. The training transcript tells us the correct word, but it does not tell us which frames correspond to **h**, **e**, or either **l**.

For an acoustic sequence of length T and a transcript of length U , the model therefore needs to learn an alignment between

$$(h_1, h_2, \dots, h_T) \quad \text{and} \quad (y_1, y_2, \dots, y_U),$$

typically with $T \gg U$. CTC and attention-based encoder-decoder models solve this problem in different ways.

9.4.2 Connectionist Temporal Classification

Connectionist Temporal Classification (CTC) allows a model to be trained from a transcript without frame-level alignment labels. At every time step, the model predicts a distribution over the output vocabulary plus a special **blank** symbol, denoted here by $\emptyset$. A blank means “emit no new token at this frame”; it is not a space between words.

A length- T sequence of frame-level symbols is called a **path** $\pi = (\pi_1, \dots, \pi_T)$. The CTC collapse function $\mathcal{B}$ converts a path to a transcript in two steps: merge consecutive repetitions of the same non-blank symbol and remove all blank symbols.

Example: A valid CTC path for “hello”

Suppose that the encoder produces ten time steps. One possible predicted path is

$$\mathbf{h}, \mathbf{h}, \mathbf{e}, \emptyset, \mathbf{l}, \emptyset, \mathbf{l}, \mathbf{l}, \mathbf{o}, \mathbf{o}.$$

Merging consecutive repetitions gives

$$\mathbf{h}, \mathbf{e}, \emptyset, \mathbf{l}, \emptyset, \mathbf{l}, \mathbf{o},$$

and removing blanks gives

$$\mathbf{h}, \mathbf{e}, \mathbf{l}, \mathbf{l}, \mathbf{o}.$$

In this case the blank between the two occurrences of **l** is necessary; without it, adjacent repeated labels would collapse into a single **l** and the result would be **h,e,l,o**.

The vocabulary may also contain a normal **space token**. In character-based systems, this token separates words; in subword systems, the word boundary is usually encoded as part of a token. Unlike the CTC blank, a space or boundary token is preserved in the final transcript. The blank is only used to construct the alignment and is removed by the collapse operation.

There are many paths to obtain the correct output, so the model is not forced to select one correct alignment. Instead, CTC adds the probabilities of **all paths** that collapse to the target transcript. This is easier to see in a very small example.

Example: Why CTC sums over paths

Suppose that there are two time steps and the target transcript is **a**. Three paths collapse to this target:

$$(\mathbf{a}, \mathbf{a}), \quad (\mathbf{a}, \emptyset), \quad (\emptyset, \mathbf{a}).$$

Assume that the model predicts

	$p(\mathbf{a})$	$p(\emptyset)$
$t = 1$	0.6	0.4
$t = 2$	0.7	0.3

Then the probability assigned to transcript **a** is

$$(0.6 \cdot 0.7) + (0.6 \cdot 0.3) + (0.4 \cdot 0.7) = 0.88.$$

CTC therefore rewards the correct transcript without requiring us to specify which time step should emit **a**.

In general, CTC assigns a transcript y the probability

$$p(y \mid x) = \sum_{\pi: \mathcal{B}(\pi)=y} \prod_{t=1}^T p_t(\pi_t \mid x),$$

and minimises the negative log-likelihood

$$\mathcal{L}_{\text{CTC}} = -\log(p(y \mid x)).$$

The product calculates the probability of one path, and the sum combines all paths that produce y . Dynamic programming computes this sum without listing every path explicitly. Figure 9.5 shows the complete pipeline.

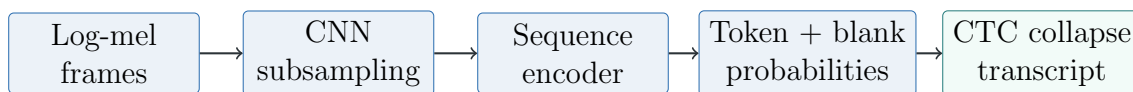


Figure 9.5: A simplified CTC-based speech-recognition pipeline.

9.4.3 Attention-Based Encoder-Decoder Models

An attention-based encoder-decoder predicts the transcript directly, one token at a time. At each step, the decoder:

1. Compares its current state with all encoder features.
2. Uses attention weights to combine the most relevant features.
3. Predicts the next token using this context and the previous tokens.

This gives the autoregressive factorisation

$$p(y \mid x) = \prod_{u=1}^U p(y_u \mid y_{<u}, x).$$

Example: Generating “cat” with attention

The decoder begins with a special start token. It may first attend to the early audio frames and predict **c**. At the next step, it receives the previous token **c**, attends to the middle frames, and predicts **a**. It then uses **c**, **a** and later frames to predict **t**, followed by an end-of-sequence token.

Unlike CTC, this decoder does not emit a token for every audio frame and does not need a blank symbol. It explicitly uses previous output tokens, which can improve linguistic consistency. The cost is that inference is sequential: the second token cannot be generated until the first one is available.

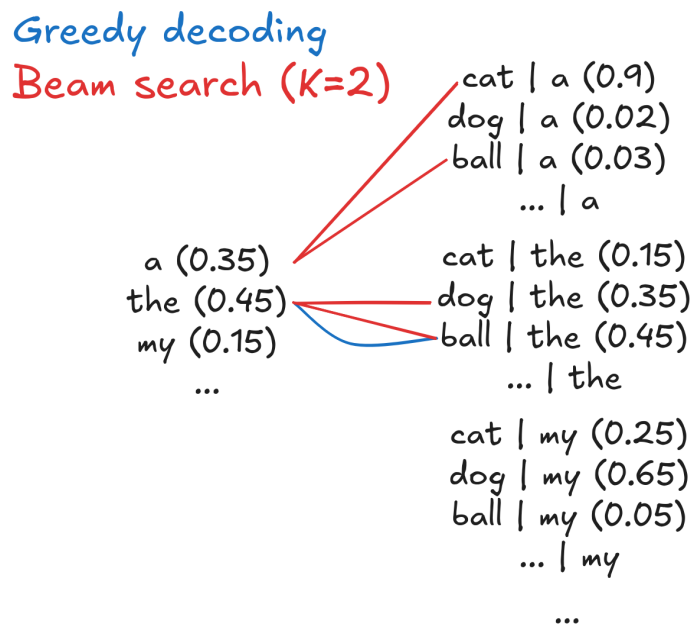
9.4.4 Decoding

At inference time, the transcript is not known and must be decoded from model predictions.

- **Greedy decoding** keeps only the most probable choice at the current step. For CTC, it selects the most probable symbol at every frame and then applies the collapse function. It is fast, but an early local choice may lead to a poor complete transcript.
- **Beam search** keeps the best K partial transcripts, called the beam. At the next step, each hypothesis is extended with possible tokens, their cumulative scores are updated, and only the best K are kept again. This repeats until decoding finishes.

Figure 9.6 shows a comparison between these methods. Greedy decoding first selects **the**, with probability 0.45, and then **ball**, with conditional probability 0.45. Its sequence score is therefore $0.45 \cdot 0.45 = 0.2025$. A beam of size $K = 2$ keeps the two best first-step options, **the** and **a**, and finds that **a cat** has the higher score $0.35 \cdot 0.90 = 0.315$.

For CTC, beam search can also combine different frame-level paths that collapse to the same transcript. A larger beam explores more alternatives but requires more computation and does not guarantee a better result. Beam search does not enumerate K^n complete paths (being n the number of steps): at each step, it expands the current hypotheses and retains only the best K partial transcripts.

Figure 9.6: Greedy decoding versus beam search with $K = 2$.

9.4.5 Word Error Rate

The standard ASR metric is **Word Error Rate (WER)**. It is based on the minimum number of word-level edits required to transform a reference transcript into the hypothesis:

$$\text{WER} = \frac{S + D + I}{N},$$

where S is the number of substitutions, D deletions, I insertions, and N the number of words in the reference. Lower values are better. WER can be greater than 1 when there are many insertions.

Example: Calculating WER

Consider

Reference: the quick brown fox jumps,
Hypothesis: the quicker brown fox jump.

There are two substitutions: quick→quicker and jumps→jump. There are no insertions or deletions, so

$$\text{WER} = \frac{2 + 0 + 0}{5} = 0.4 = 40\%.$$

WER depends on tokenisation and text normalisation. Capitalisation, punctuation, numbers, and contractions should therefore be processed consistently before systems are compared. WER also measures exact word edits, not whether two sentences have the same meaning.

9.5 Exercises

Exercise 1: Waveforms and Sampling

A batch contains 16 stereo recordings, each lasting three seconds at 16 kHz. What is the shape (B, C, L) of the waveform tensor? What is the Nyquist frequency?

Exercise 2: Frames and Hops

At a sample rate of 16 kHz, calculate the number of samples in a 25 ms window and a 10 ms hop. Briefly explain what each of these two parameters controls.

Exercise 3: Building Log-Mel Features

A mono spectrogram has shape $(1, 201, 98)$. What is its shape after applying 80 mel filters? Briefly explain why mel filters and a logarithmic scale are used.

Exercise 4: Safe Audio Augmentation

For each of the following tasks, discuss whether pitch shifting and time stretching preserve the label: environmental sound classification, pitch estimation, speaker verification, and temporally localised event detection.

Exercise 5: Spectrograms and Images

Explain why a 2D CNN can process a spectrogram and why the frequency and time axes should nevertheless not be treated exactly like the two spatial axes of a natural image.

Exercise 6: Choosing a Model Output

Compare a model that classifies a complete audio clip with one that transcribes speech. In which case could the CNN features be pooled into one vector, and in which case should their temporal order be preserved?

Exercise 7: CTC Collapse

Apply the CTC collapse operation to $c, c, \emptyset, a, a, \emptyset, t, t$. Then construct a different valid CTC path that produces the same word.

Exercise 8: CTC and Attention

Briefly explain how CTC and an attention-based decoder solve the alignment between audio frames and text tokens. Mention one practical difference during inference.

Exercise 9: Greedy Decoding and Beam Search

Why can beam search find a better transcript than greedy decoding? What is the main cost of increasing the beam size K ?

Exercise 10: Word Error Rate

Compute the WER for the reference “a small red car” and the hypothesis “a red fast car”. Identify the substitutions, deletions, and insertions used by your minimum-edit alignment.